TECHNICAL INTERVIEW PREP · 6 DE 10

Technical Interview - Boilerplate 2 - Node + Serverless + AWS + Github Actions

Giuliana Olmos · 21 de noviembre de 2024 · https://dev.to/giulianaolmos/technical-interview-boilerplate-2-node-serverless-aws-github-actions-5apf

Introduction

Hello everyone!! 🤗

I want to share another boilerplate I prepared for take-home interviews. A few weeks ago, we talked about a boilerplate with Node.js, Express, and PostgreSQL.✨Today's boilerplate is a bit different. This time, I'm using Node.js, Express, TypeScript, and a NoSQL database✨. Everything will be deployable on AWS using Lambda and DynamoDB through the Serverless Framework. I'll also touch on setting up GitHub Actions for CI/CD.

This boilerplate is different because it allows interviewers to run live tests. We can use AWS's free tier, which makes it cheaper than using a server and SQL database like before. Plus, it gives us a chance to show some real-world deployment experience. 🤓

The main idea is to create a serverless CRUD connected to a NoSQL database.

If you create both boilerplates, you'll be able to handle about 90% of the take-home assignments companies usually ask for. You'll be ready for projects that need either an SQL or NoSQL database.

You'll also have examples that show you can work with autoscaling services, microservices, or even monolithic apps.

If I come across another take-home with a different approach, and it's something that can be reused, I'll create another boilerplate and share it in this post.

Now, let's get into the project!

Like in the last boilerplate, I won't explain every step to build it. I'll just talk about some important points.

Topics

- Before to start...
- Serverless.yml
- DynamoDB and Dynamoose
- Lambdas
- Tests

- Github Actions
- Final Steps
- The End

Before to start...:

What is ✨Serverless✨? And is it the same as the ✨Serverless Framework✨?

Serverless is a cloud computing model where the cloud provider automatically manages the infrastructure and scales applications as needed. This allows you to focus on writing code, while the cloud provider handles server provisioning, scaling, and management. In other words, you don't need to worry about DevOps or managing the infrastructure.

However, **Serverless** is not the same as the **Serverless Framework**.

The **Serverless Framework** is a specific tool that helps developers build and deploy serverless applications across different cloud platforms. In my case, I will be deploying to AWS, as it's the platform I have the most experience with. Google Cloud Platform (GCP) is also a good option.

To work with AWS and the Serverless Framework, you'll need to:

1. Set up your AWS credentials on your computer.
2. Install the Serverless Framework globally by running:

```
npm i serverless -g
```

Serverless.yml:

Serverless Framework v4 allows us to define all the infrastructure in the `serverless.yml` file.

Mine looks something like this:

```
app: boilerplateservelesslambadadynamo
service: boilerplate-serverless-Lambda-Dynamo

provider:
  name: aws
  runtime: nodejs18.x
  region: us-east-1
  iamRoleStatements:
    - Effect: Allow
      Action:
        - dynamodb:CreateTable
        - dynamodb:DescribeTable
```

```
                - dynamodb:PutItem
                - dynamodb:UpdateItem
                - dynamodb:DeleteItem
                - dynamodb:GetItem
                - dynamodb:Scan
                - dynamodb:Query
              Resource:
"arn:aws:dynamodb:${self:provider.region}:*:table/${self:provider.environment.TABLE_NAME}"
          environment:
            TABLE_NAME: Users

        package:
          individually: true

        functions:
          createUser:
            handler: src/lambdas/createUser.handler
            events:
              - http:
                  path: /users/create
                  method: post
                  cors: true

          getUsers:
            handler: src/lambdas/getUsers.handler
            events:
              - http:
                  path: /users/getUsers
                  method: get
                  cors: true

          deleteUser:
            handler: src/lambdas/deleteUser.handler
            events:
              - http:
                  path: /users/deleteUser
                  method: delete
                  cors: true

          updateUser:
            handler: src/lambdas/updateUser.handler
            events:
              - http:
                  path: /users/updateUser
                  method: put
                  cors: true

          createUsersTransaction:
            handler: src/lambdas/createUsersTransaction.handler
            events:
              - http:
                  path: /users/createUsersTransaction
                  method: post
                  cors: true

        resources:
          Resources:
            UsersTable:
              Type: AWS::DynamoDB::Table
              Properties:
                TableName: Users
```

```
            AttributeDefinitions:
              - AttributeName: userId
                AttributeType: S
            KeySchema:
              - AttributeName: userId
                KeyType: HASH
            BillingMode: PAY_PER_REQUEST

    plugins:
      - serverless-offline
```

Provider:

The `provider` section defines the cloud platform and configurations for deploying your serverless application.

Package: Individually: true :

I set this configuration to package and deploy each Lambda function individually, rather than bundling them all together. This is useful because it reduces deployment times and improves performance by packaging only the relevant code for each function.

Functions:

In this section, we define each Lambda function, the handler we will use, and the event that triggers the Lambda. Since this is an HTTP CRUD application, the Lambdas will be triggered by HTTP calls. Make sure to enable CORS, otherwise, you might run into issues when consuming the endpoints from a frontend.

Resources:

The `resources` section is used to define additional AWS CloudFormation resources that the Serverless Framework doesn't natively support. In my case, I declared a DynamoDB table with `userId` as the key. You can also declare other resources like S3 buckets, SQS queues, and more if needed.

Plugins:

At the end, there's a list of plugins you can install. In my case, I installed a plugin to test my Lambdas locally without needing to redeploy them after every code change.

DynamoDB and Dynamoose:

To be honest, when I code take-home projects from scratch, I don’t include too many extra packages because it means spending more time on the project. However, since I had time to prepare a nice boilerplate, I decided to use ✨Dynamoose✨.

Why Dynamoose?

Working directly with DynamoDB can be a nightmare. It has one of the worst documentations I've ever read — and I've worked with the App Store Connect API documentation before! 🤣

Dynamoose is based on Mongoose, so it offers a similar experience but for DynamoDB. It helps us work faster and more clearly with the database.

Example Schema

For this example, I created a very simple schema for a user. This schema is then used in a controller to handle all the actions for the Lambdas.

```
import * as dynamoose from "dynamoose";

const UserSchema = new dynamoose.Schema({
  userId: {
    type: String,
    hashKey: true,
  },
  name: String,
  email: String,
});

export const User = dynamoose.model(process.env.TABLE_NAME!, UserSchema);
```

Transactions

As I mentioned earlier, I've included an example of transactions. Sometimes it may be necessary to implement a transaction in a take-home project, and I don't want to waste too much time learning how to implement it during the process.

```
//https://dynamoosejs.com/guide/Transaction
 async createUsersByTransaction(usersData: { name: string; email: string }[]) {
   try {
     const transactionItems = usersData.map((userData) => {
       return User.transaction.create({
         userId: uuidv4(),
         ...userData,
       });
     });

     await dynamoose.transaction(transactionItems);
   } catch (error) {
     throw new Error(`Failed to create Users by transaction ${error}`);
   }
 }
```

Lambdas:

The Lambdas are written in TypeScript. Luckily, Serverless Framework version 4 builds TypeScript automatically. If you're using an older version of Serverless, you may need to install:

```
npm i --save-dev serverless-plugin-typescript
```

Then, you'll need to add it as a plugin:

```
plugins:
  - serverless-plugin-typescript
```

Here's what my Lambda function to create a user looks like:

```
import { APIGatewayProxyEvent, APIGatewayProxyResult } from "aws-lambda";
import { UserController } from "../controller/userController";

export const handler = async (
  event: APIGatewayProxyEvent
): Promise<APIGatewayProxyResult> => {
  const userData = JSON.parse(event.body || "{}");
  try {
    const userCtrl = new UserController();
    const response = await userCtrl.createUser(userData);
    console.log("User created:", response);
    console.log({ userData });
    return {
      statusCode: 200,
      body: JSON.stringify({ message: `User created`, response }),
    };
  } catch (error) {
    console.error("Error creating User:", error);
    return {
      statusCode: 500,
      body: JSON.stringify({ message: "Failed to create User" }),
    };
  }
};
```

After putting all the logic into my controllers, everything looks pretty much the same for each Lambda. 😄

> **Important Tip** Remember to export the handler, otherwise, when you reference it in `serverless.yml`, it won't be able to find the module.

```
createUser:
   handler: src/lambdas/createUser.handler
   events:
```

```
            - http:
                path: /users/create
                method: post
                cors: true
```

> **Extra Info** If you're interested, I also have an example from a technical challenge where I had to deliver an app deployed in AWS with both a frontend and backend.

> For the frontend, I used AWS Amplify: LoanPro Frontend - React.

> For the backend, I used the Serverless Framework: LoanPro Backend - Serverless.

Tests:

One thing I've noticed a lot in take-home projects (since I used to review them) is that many people don’t add tests to their code. I understand that sometimes we don’t have enough time, so we focus more on the business logic. BUT, a good project is one that already has tests.

It's not necessary to work with TDD, but the code should still have tests.

It's also important to know how to mock services. Having this experience will help you in future take-home projects. If you take the time to write tests and mock services now, the next time you’ll be able to code them much faster.

For this project, I tested my code using Jest (mainly because I've used Jest throughout my experience).

Github Actions:

I wanted to try GitHub Actions to automate my deployments, so I took advantage of GitHub’s free tier to deploy this project.

For this type of deployment, where GitHub doesn’t need to host a frontend and you only use Actions, the free tier offers around 2,000 minutes per month.

In this project, each deploy takes about 1 minute.

This is my `.github/workflows/main.yml` file.

```
        # Documentation: https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-
github-actions
        name: Deploy to AWS using Serverless Framework

        on:
          release:
```

```
    types: [created]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout the repository
        uses: actions/checkout@v2

      - name: Setup Node.js
        uses: actions/setup-node@v2
        with:
          node-version: "18"

      - name: Install dependencies
        run: npm install

      - name: Build TypeScript code
        run: npm run build

      - name: Deploy with Serverless
        run: npx serverless deploy
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          SERVERLESS_ACCESS_KEY: ${{ secrets.SERVERLESS_ACCESS_KEY }}
```

I set it to run every time I create a release. I recommend this option instead of deploying with every commit because, if you can deploy manually or test offline, it won't be necessary to use up your free tier minutes unnecessarily.

An important point is that, for this automatic deployment, you only need to set some credentials.

```
AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
SERVERLESS_ACCESS_KEY: ${{ secrets.SERVERLESS_ACCESS_KEY }}
```

You set these credentials directly in the same GitHub repository. To do this, navigate to **Settings** > **Secrets and variables** > **Actions** in your GitHub repository to add the necessary secrets.

Final Steps:

This boilerplate is configured to be deployed either using GitHub Actions or manually via the console.

For GitHub Actions, as I mentioned before, you can trigger the deployment by creating a release.

Manual Deployment

If you prefer to deploy it manually without using CI/CD, you'll need to set up your AWS credentials on your computer and install the Serverless Framework (as I mentioned earlier).

Once that’s done, just run the following commands:

```
npm i
npm run build
```

Then, deploy your stack with:

```
serverless deploy
```

This will deploy the entire stack of your project, and you'll get the exposed endpoints for each Lambda function in return.

Running Locally

After the first deployment, if you don't want to redeploy every time you test a function, I’ve also installed a plugin in the `serverless.yml` file to run the Lambdas locally. This exposes the endpoints at `localhost:3000`.

To run locally, just follow these steps:

```
npm i
npm run build
serverless offline
```

This will expose the endpoints and allow you to monitor your logs directly in the project console.

Sharing Your Live Code

Deploying your project allows you to send a Postman collection to your interviewers so they can interact with your live code. This way, they won’t need to run the server on their machines. Just remember to delete the stack afterward to avoid any malicious use of your project.

Why Not Docker?

I chose not to use Docker for this project to avoid unnecessary complexity. When using the Serverless Framework and the AWS free tier, Docker isn’t needed. You can always add Docker if you want, but it’s not necessary.

In my experience working in FinTech, we used this setup for the entire serverless architecture, and the absence of Docker was never an issue.

The End:

I really hope this boilerplate and the explanation are useful to you in your job search. 🫶

I'll leave the link to my repo below: https://github.com/GiulianaEOlmos/boilerplate-node-lambdas-dynamodb

You are free to clone and use it for your take-home projects if you want, and if you receive any feedback about it, I would love to hear it! BUT, as I mentioned in the previous boilerplate, you should still try to create your own. It's a great way to learn how to configure Serverless and GitHub Actions, and you'll be able to explain it confidently if an interviewer asks about it.

This time, I added a lot of extra comments from my previous experiences, and I hope those were helpful to you as well.

See you next week, and I wish everyone the best of luck in their job search! We'll get that dream job! 💖

Bye bye!